Project 1 Part 1

Important:

- Write your name as well as your NU ID on your assignment. Please number your problems.
- Submit both results and your code.
- Give complete answers. Do not just give the final answer; instead show steps you went through to get there and explain what you are doing. Do not leave out critical intermediate steps.
- This assignment must be submitted electronically through Gradescope by March 9th 2026 (Monday) by 11:59 PM.
- All of your codes must be commented.
- All of your plots must be labeled, titled and have a legend (when applicable).

This assignment serves as the first part of a larger project, focused on healthcare, that you will be completing in this class. The project consists of developing a (simplistic) machine learning model that allows you to predict the diastolic blood pressure and systolic blood pressure of a patient given the total cholesterol level in their blood. In this part of the project, you will explore the gradient descent algorithm and start implementing the framework needed to build the predictive model.

The main goals of this part of the project are the following:

- Understand what are derivatives and how they can be used to optimize functions.
- Learn how one optimization algorithm (gradient descent) works.
- Gain a a deeper understanding of the limitations of gradient descent works by analyzing its performance on different functions.
- Learn how to develop code to be able to implement the mathematical concepts presented in this project. The project will guide you to divide your code into several smaller functions. You should always avoid having functions with a large number of code lines. It is much easier to read code that is split into several functions rather than having to read one large block of code. In this part of the project, I am walking you through how to divide your code into several functions that only include a small number of lines. In the next part, you will divide your code into several functions by yourselves.
- Implement derivative approximation to gradient descent to any function.
- Generalize gradient descent to minimize functions of two variables.

1 Derivatives and Optimization

In many applications, optimizing (maximizing or minimizing) functions is needed. For example:

- In Economics, consumers seek to **maximize** their utility, while firms want to **maximize** their profit.
- In Machine Learning, building an accurate predictive models requires **optimizing** the fit of model to data.

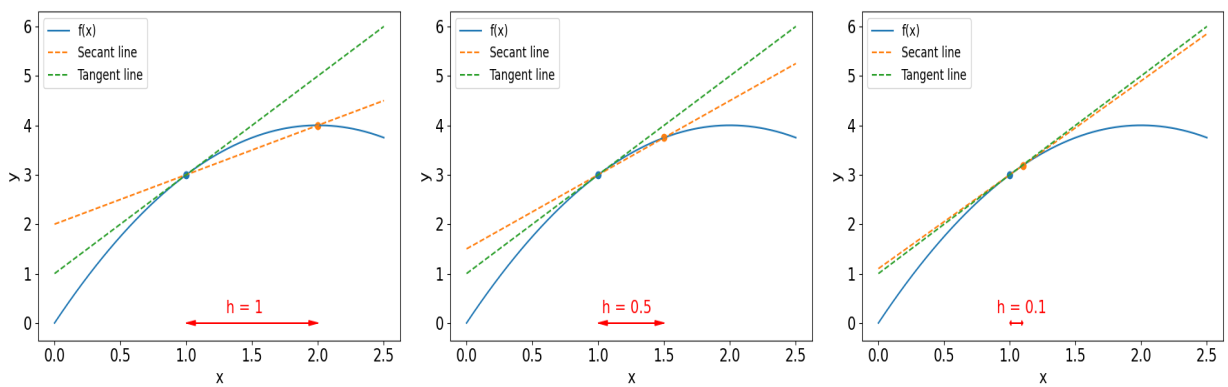
- In Civil Engineering, **optimization** is used in many problems such as improving traffic management, water resource allocations or even resource leveling.
- In Medicine, mathematical models are used to plan surgical procedures, **optimize** the scheduling of appointments, surgeries, and other medical procedures to **minimize** wait times and **maximize** resource utilization.

Optimizing functions require the computation of a derivative $f'(x)$ of a function $f(x)$. Another notation that can be used for a derivative $f'(x)$ is $\frac{df}{dx}(x)$. We first provide you with a background on derivatives.

The derivative $f'(x)$ computes the slope of the tangent line to $f(x)$ at some point x. In figure 1, we plot some function $f(x)$, the tangent line at $x = 1$, and the secant line joining the points $(1, f(1))$ and $(1 + h, f(1 + h))$ for some $h > 0$. The secant line's slope is given by,

$$\frac{f(x+h) - f(x)}{(x+h) - x} = \frac{f(x+h) - f(x)}{h}.$$

Notice that as h becomes smaller and approaches 0, the secant and tangent lines become almost confounded. When $h \to 0$, the secant line becomes the tangent line. This suggests that the slope of the secant line, as $h \to 0$, should be equal to the slope of the tangent line to $f(x)$ which we defined as $f'(x)$.



(a) $h = 1$. (b) $h = 0.5$. (c) $h = 0.1$.

Figure 1: Plot of $f(x)$, tangent line at $x = 1$ and secant line joining $(1, f(1))$ and $(1+h, f(1+h))$ for different values of h.

More formally, we can then write,

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}. \tag{1}$$

Let's compute some derivatives. We'll take as example $f(x) = 2x$. Then,

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} = f'(x) = \lim_{h \to 0} \frac{2(x+h) - 2x}{h} = \lim_{h \to 0} \frac{2x + 2h - 2x}{h} = \lim_{h \to 0} \frac{2h}{h} = \lim_{h \to 0} 2 = 2.$$

This means that the derivative of the linear function $f(x) = 2x$ is $f'(x) = 2$.

As practice, show the statements below. You do not need to submit your work for these exercises, this is just to have you practice computing derivatives

- The derivative of $f(x) = a$ is $f'(x) = 0$ for any real number a.
- The derivative of $f(x) = ax$ is $f'(x) = a$ for any real number a.
- The derivative of $f(x) = ax^2$ is $f'(x) = 2ax$ for any real number a.

- The derivative of $f(x) = ax^2 + ax$ is $f'(x) = 2ax + a$ for any real number a.
- The derivative of $f(x) = g(x) + p(x)$ is $f'(x) = g'(x) + p'(x)$ for any functions g and p. This means that the derivative of a sum of functions is the sum of their derivatives.

2 Gradient Descent

Suppose you want to minimize a function $f(x)$. A minimum of $f(x)$ occurs at a point x where the derivative $f'(x)$, i.e slope of the tangent line to $f(x)$, is 0.

For example, consider $f(x) = x^2$. If you plot the function, you can see that its minimum value is achieved at $x = 0$. We can also find the location of the minimum by computing the derivative $f'(x) = 2x$ and solving $f'(x) = 2x = 0$ resulting in the critical point $x = 0$. For $f(x) = x^2$, we can conclude that the minimum of $f(x)$ is achieved at $x = 0$ and has a value of $f(0) = 0$.

At times, it might be challenging to solve the equation $f'(x) = 0$, depending on how complex $f'(x)$ is. For example, if we consider $f(x) = \frac{e^x}{(1+x^2)^2}$, the equation $f'(x) = 0$ cannot be solved exactly. We need some numerical algorithms that would allow us to estimate the minimum of a function. One possible optimization algorithm is known as the **gradient descent algorithm**.

We motivate the idea behind gradient descent as follows: suppose after hiking up a mountain, you finally reached the top and seek to go down the mountain to get to the ground. How can you do that in the fastest way possible? We define $f(x)$ to be the function modeling the mountain. You can notice that the fastest direction to go down the mountain is to always follow the direction of the derivative of that function at your current location. You can see that in figure 2.

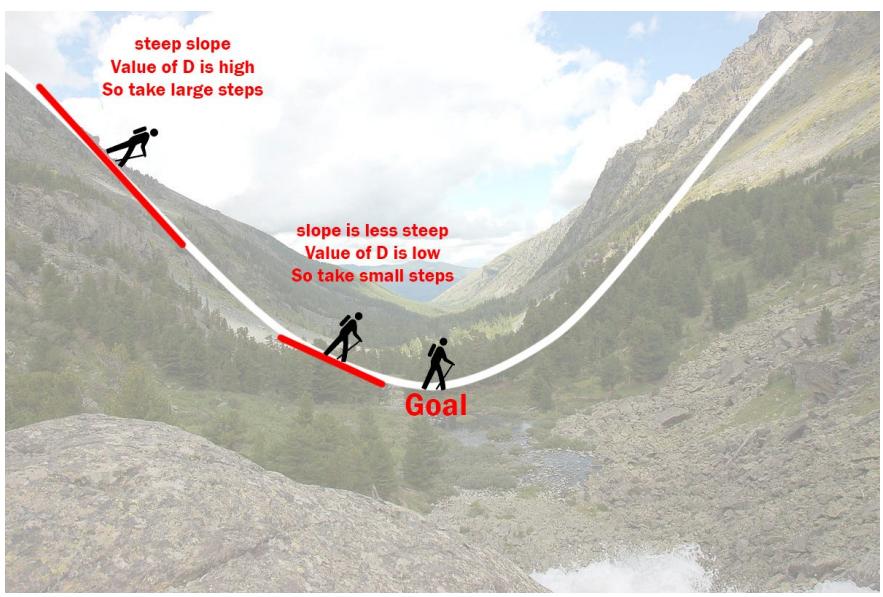


Figure 2: Gradient Descent Algorithm. D is the value of the magnitude of the derivative.

You can take a step in the direction of the derivative to go down as fast as possible from the mountain. The next question you might ask yourself though is how big should your step be? Usually, a smaller step size is preferred over a larger step size in order not to miss the location of minimum and climb up the mountain further. Let α be the step size.

The gradient descent algorithm would consist of the following steps:

- You start at some initial point x_0 on the mountain modeled by $f(x)$.

- You keep on taking steps of size α in the direction of the derivative. That is, you update your position by applying,

$$x_1 = x_0 - \alpha f'(x_0), \tag{2}$$

where x_1 is the new position. The equation above can be generalized to the kth step as,

$$x_{k+1} = x_k - \alpha f'(x_k). \tag{3}$$

- You keep on updating the position according to the gradient descent update rule up until the value of x_k barely changes. That is, for some number of steps k, you would notice that x_{k+1} and x_k are very close to each other up to some tolerance ϵ. For instance, say you chose the tolerance $\epsilon = 0.001$, a relatively small number. You would keep on applying the gradient descent update rule up until x_{k+1} and x_k are within $\epsilon = 0.001$ from each other, i.e.,

$$|x_{k+1} - x_k| < \epsilon. \tag{4}$$

2.1 Quadratic Functions

We consider the following two quadratic functions,

$$f_1(x) = x^2 \text{ and } f_2(x) = x^2 - 2x + 3. \tag{5}$$

1. Start by writing a function that implements the gradient descent algorithm. In the algorithm, define a variable *iter* that count the number of gradient descent iterations performed. In the while loop, add a second condition that checks whether *iter* is not greater than some maximum number of iterations allowed, given by *iter_max*. This will allow you to avoid infinite loops. In all parts of this problem, you can fix *iter_max* = 1000. Test your algorithm for $x_0 = 3$, $\alpha = 0.1$ and $\epsilon = 0.001$ for both $f_1(x)$ and $f_2(x)$. Plot the location of the optimal x you find using gradient descent along with $f_1(x)$ and $f_2(x)$ (each function and its optimizer on one separate plot).

By the end of this part, you should have implemented, including writing comments, the following functions:

```
def gradient_descent():

def f1():

def deriv_f1():

def f2():

def deriv_f2():

def plot_opt():
```

The inputs of the functions are omitted above as I want you to figure out the inputs of the functions.

In the next sub-parts, we will be conducting a perturbation analysis of the parameters, also known as parameter hyper tuning, to understand the effect of varying the values of x_0, $\alpha$and ϵ on the output of the gradient descent algorithm.

2. We vary the value of x_0, and fix $\alpha = 0.1$ and $\epsilon = 0.001$. We consider $x_0 = 3$ and $x_0 = -3$. For each value of x_0, compute the optimal value of x_k obtained by the gradient descent algorithm for both $f_1(x)$ and $f_2(x)$. Compare the optimal values computed for both values of x_0. Could you have expected how the optimal result varies for different x_0 before even running the algorithm?

3. Now, we fix $x_0 = 3$ and $\epsilon = 0.001$ and consider different values of $\alpha \in \{1, 0.001, 0.0001\}$. What is the output of the gradient algorithm for every value of α? Are you observing any variations in the output? If so, why do you think this is happening?
Hint: For $\alpha = 1$, check the value of *iter*.

4. Finally, we fix $x_0 = 3$ and $\alpha = 0.1$, and consider the following values of $\epsilon \in \{0.1, 0.01, 0.0001\}$. Once again, what is the output of the gradient algorithm for every value of ϵ? Are you observing any variations in the output? If so, why do you think this is happening?

When testing the gradient descent implementation with different values of x_0, α and ϵ, include all of these tests within one main function:

```
def main():
```

2.2 More Complex Functions

We now consider a more complex function given by,

$$f_3(x) = \sin(x) + \cos(\sqrt{2}x), \text{ for } 0 \leq x \leq 10. \tag{6}$$

1. Start by plotting $f_3(x)$. Notice how many local minima $f_3(x)$ has. For this part, you will need to write one function:

```
def plot_f3():
```

2. We fix $\alpha = 0.1$ and $\epsilon = 0.0001$. The derivative of $f_3(x)$ is given by $f_3'(x) = \cos(x) - \sqrt{2}\sin(\sqrt{2}x)$. Implement the gradient descent algorithm for $x_0 = 1$, $x_0 = 4$, $x_0 = 5$ and $x_0 = 7$. What is the output of the gradient descent algorithm for each value of x_0? Plot the function along with the local minima you obtained for the different values of x_0.

   You can update the function *plot_ opt* you implemented in part 1 of section 1.1 to plot more than one optimal points. After completing this part, you should have coded (or modified) the following additional functions:

```
def plot_opt():

def f3():

def deriv_f3():
```

3 Derivative Approximation for Functions of One Variable

For complex functions $f(x)$, computing the derivatives can be challenging. In many algorithms, computer scientists come up with ways to approximate derivatives. Notice that the quantity $\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$ will be very close to the derivative $f'(x)$ for a small value of h.

1. Write a Python function which approximates the derivative of any function $f(x)$ at some point x. You will need to choose a small value of h. Test your Python function on $f_1(x) = x^2$ and $f_1(x) = x^2 - 2x + 3$. Check that your function is returning similar values to the exact derivatives for different values of x. You can choose whichever values of x you want to test.

2. Modify the *gradient_ descent()* function that you implemented in the previous assignment so that it uses the derivative approximation function instead of the actual derivative. Test your algorithm on $f_1(x) = x^2$ and $f_1(x) = x^2 - 2x + 3$ for $x_0 = 3$, $\alpha = 0.1$ and $\epsilon = 0.001$. Do you still obtain the same results as in your original implementation?

4 Gradient Descent for Functions of Two Variables

We extend the gradient descent framework to functions of two variables, for instance function $f(a, b)$ where a and b are its inputs. Multi-variable functions occur very often in a variety of applications. For instance, suppose you are driving on some highway and your car is currently at position x. You are interested in computing the car's future position after some time t when driving with speed v. You can see that the car's future position depends on several variables: x, t and v.

Suppose you are interested in minimizing a two-variable function $f(x, y)$. The gradient descent algorithm can still be applied and its logic remains the same:

- You start at some initial points x_0 and y_0.
- You keep on taking steps of size α in the direction of the derivative with respect to each variable. That is, you update the values of x_0 and y_0 by applying,

$$x_1 = x_0 - \alpha \frac{df}{dx}(x_0, y_0) \text{ and } y_1 = y_0 - \alpha \frac{df}{dy}(x_0, y_0) \tag{7}$$

  where x_1, y_1 are the updated values of the parameters, $\frac{df}{dx}(x_0, y_0)$ is the derivative of f with respect to x computed at (x_0, y_0) and $\frac{df}{dy}(x_0, y_0)$ is the derivative of f with respect to y computed at (x_0, y_0).

  The intuition behind derivatives of multi-variable functions is the same as with single variable functions. For example, suppose you want to compute the derivative of f with respect to x at some point (x_0, y_0), you can compute the slope of the "line" joining the points (x, y) and $(x + h, y)$ and take $h \to 0$. You can write,

$$\frac{df}{dx}(x_0, y_0) = \lim_{h \to 0} \frac{f(x_0 + h, y_0) - f(x_0, y_0)}{h} \text{ and } \frac{df}{dy}(x_0, y_0) = \lim_{h \to 0} \frac{f(x_0, y_0 + h) - f(x_0, y_0)}{h}. \tag{8}$$

  The derivatives can be approximated by extending the approximation function you developed in part 1 of section 1. The gradient descent update rule can be generalized to the kth step as,

$$x_{k+1} = x_k - \alpha \frac{df}{dx}(x_k, y_k) \text{ and } y_{k+1} = y_k - \alpha \frac{df}{dy}(x_k, y_k) \tag{9}$$

- You keep on updating the values x_k and y_k according to the gradient descent update rule up until the value of both x_k and y_k barely change. That is, for some number of steps k, you would notice that x_{k+1} and x_k, as well as y_{k+1} and y_k, are very close to each other up to some tolerance ϵ. For instance, say you chose the tolerance $\epsilon = 0.001$, a relatively small number. You would keep on applying the gradient descent update rule up until,

$$|y_{k+1} - y_k| < \epsilon \textbf{ and } |y_{k+1} - y_k| < \epsilon. \tag{10}$$

1. Extend the function written in part 1 of section 1 to approximate derivatives of functions of two variables.

2. Implement the gradient descent algorithm for functions of two variables. Approximate the derivatives in the update steps using the function you wrote in the previous part. Make sure your code is structured into several functions as we did in the previous assignment. Test your code on $f(x, y) = x^2 + y^2$ with $x_0 = 3$, $y_0 = 3$, $\alpha = 0.1$ and $\epsilon = 0.001$. Plot the function with the minimum obtained. Note that this will be a $3D$ plot.

5 Report

Write a report that explains how you designed your code to implement gradient descent for one variable first and how you generalized it to several variables. Explain how you tested your code with different cases. Summarize what you learnt when applying a perturbation analysis of the parameters, also known as parameter hyper tuning, and how different values of x_0, α and ϵ can affect the performance of gradient descent.